FASTAPI DECEPTION ARCHITECTURE

الدليل الماستر الشامل ومعمارية المنظومة الدفاعية

Enterprise Active Threat Deception, Sub-Millisecond Session Isolation, and Tamper-Evident Digital Forensics Extraction

المرجع المعماري والتنفيذي الموحد لمشروع التخرج: هندسة الدفاع السيبراني النشط، توجيه حركة المرور غير المتزامن، مجمع حاويات العزل السريعة، وسلسلة العهدة الرقمية المقاومة للتلاعب.

قائد الفريق والمهندس المعماري
Team Leader & System Architect — (Sadjad) سجاد

مهندس البوابة وتصنيف المرور
Backend & Deception Gateway Engineer — (Ali) علي

مهندس التطبيق الحقيقي والبيانات
Core Application & Data Engineer — (Yousif) يوسف

مهندس الشراك والعزل والجنائيات
Honeypot, Sandbox & Forensics — (Ammar) عمار Engineer

تاريخ الإصدار والتحقق
Production Verified // 2026 نوفمبر

حالة الاختبارات والاعتماد
Passed (5 of 5 Test Suites Active) 100%

1. الملخص التنفيذي والأهداف الاستراتيجية للمشروع

📌 **الرؤية الهندسية للمشروع (Core Engineering Vision)**

مشروع **معمارية الخداع السيبراني المبنية على FastAPI لكشف التهديدات واستخراج الأدلة الجنائية** هو نظام دفاع سيبراني نشط (Active Cyber Defense) تم تصميمه وفق معايير المؤسسات الكبرى (Enterprise-Grade). يهدف إلى نقل المؤسسة من الدفاع السلبي التقليدي إلى استدراج المهاجمين واحتجازهم داخل بيئات وهمية معزولة شبكياً واستخراج أدلة إدانتهم بدقة قانونية قطعية.

1.1 المشكلة الهندسية في منظومات الحماية التقليدية

تعتمد منظومات الحماية الشائعة مثل الجدران النارية (Firewalls / WAF) وأنظمة كشف ومنع التسلل (IDS / IPS) على مبدأ **الحظر المباشر (Direct Drop / Block)** عند رصد أي سلوك مشبوه. هذا النهج يمتلك ثغرات استراتيجية:

- **إشعار المهاجم بكشفه (Attacker Alerting)**: عندما يستلم المهاجم رد `403 Forbidden` أو تنقطع جلسته، يدرك فوراً أن أسلوبه أو عنوان IP الخاص به قد تم وسمه، فيلجأ لتغيير عنوانه أو تعديل تقنيات الهجوم لتجاوز الجدار الناري.
- **فقدان الاستخبارات الأمنية (Loss of Threat Intelligence)**: لا يتيح الحظر للمدافعين معرفة نوايا المهاجم النهائية، أو أدوات استغلال الثغرات (Exploits) التي يمتلكها، أو عناوين سيرفرات القيادة والسيطرة (C2) التي يحاول الاتصال بها.
- **كثرة الإنذارات الكاذبة (High False Positive Rates)**: تؤدي قواعد الفحص الفضفاضة في WAF إلى حظر مستخدمين شرعيين، مما يعطل أعمال المؤسسة.

1.2 الحل المبتكر: الدفاع بالخداع التفاعلي (Interactive Deception Architecture)

يقوم مشروعنا على قلب المعادلة: بدلاً من حظر المهاجم، نقوم بما يلي:

1. **حقن الشراك الخداعية الموقعة (HMAC-SHA256 Signed Lures)**: توليد ترويسات HTTP مزيفة (مثل `X-Internal-Debug-Token`) وروابط مخفية في JSON (مثل `/api/v1/internal/vault-keys`) وتضمينها داخل استجابات التطبيق الحقيقي.
2. **التحويل التلقائي والشفاف (Transparent Sub-Millisecond Routing)**: عند محاولة المهاجم أو أداة الفحص استغلال هذا الطُعم، تقوم البوابة `deception_gateway` بتصعيد حالته فوراً وتحويل جلسته إلى **بيئة عزل ديناميكية مُحمّاة مسبقاً (Prewarmed Sandbox)** دون انقطاع اتصاله.
3. **محاكاة البيئة الواقعية (High-Interaction Emulation)**: يتفاعل المهاجم مع نظام ملفات لينكس وهمي وشل تفاعلي وخزائن مفاتيح مصطنعة، مما يستنزف وقته وموارده ويوهمه بأنه سيطر على السيرفر.
4. **التحقيق الجنائي الرقمي اللحظي (Automated DFIR)**: تسجيل كل أمر، وحزمة شبكة (PCAP)، ونداء نواة (eBPF)، وربطها بسلسلة عهدة رقمية مشفرة (Tamper-Evident Chain of Custody) مقبولة قانونياً.

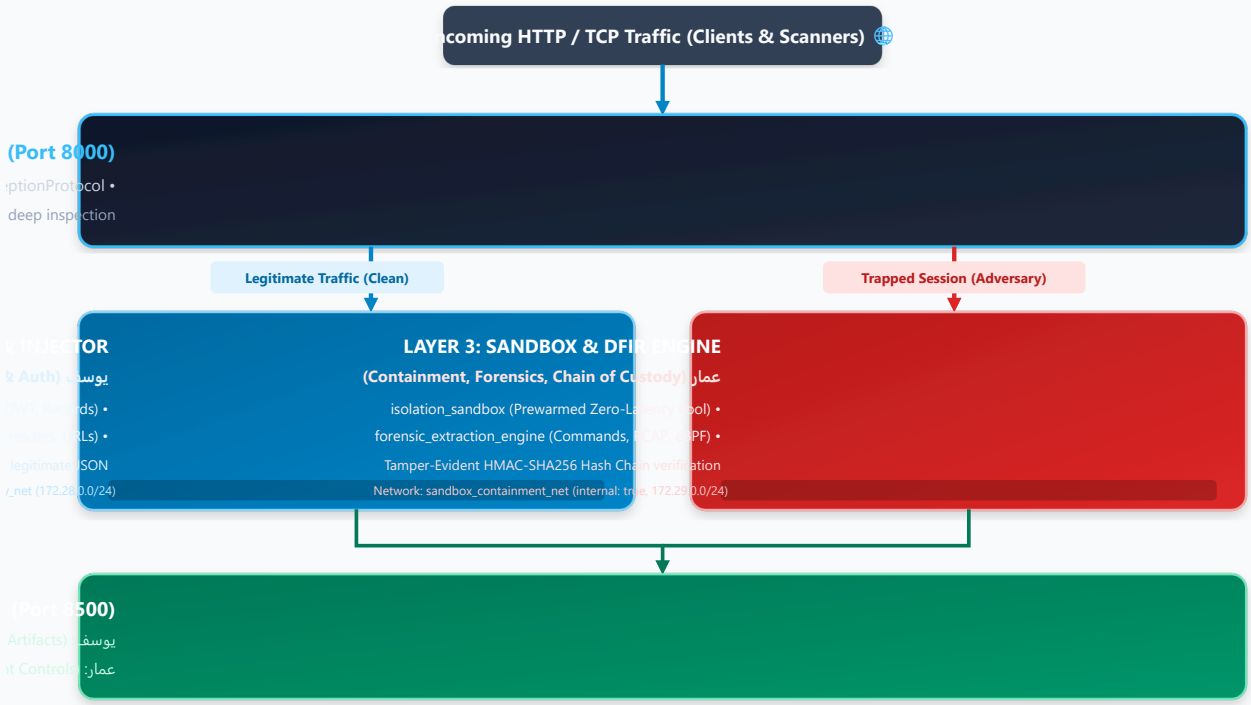
1.3 بطاقة الهوية التقنية للمشروع (Technology Stack)

المكون (Component)	التقنية المستخدمة (Technology)	الإصدار / المواصفة	الدور المعماري في المشروع
Core Framework	FastAPI (Async/Await)	=< 0.110.0	بناء الخوادم غير المتزامنة عالية الأداء للطبقات الأربع

المكون (Component)	التقنية المستخدمة (Technology)	الإصدار / المواصفة	الدور المعماري في المشروع
ASGI Server	Uvicorn Standard	>= 0.29.0	تشغيل الخوادم الخمسة مع بروتوكول L4 الخادع المخصص
Data Validation	Pydantic v2 Core	>= 2.7.0	توحيد عقود البيانات الصارمة في `shared_contracts`
Session & Streams	Redis 7 & fakeredis	>= 5.0.0	تخزين حالات الجلسات فائقة السرعة وبث التنبيهات عبر Streams
Relational Database	SQLAlchemy 2.0 + AsyncPG	>= 2.0.30	التخزين الجنائي وسجلات الإنتاج مع Fallback إلى aiosqlite
Containerization	Docker & Docker Compose	Engine v25+	عزل الشبكات الثلاث وحاويات الخداع المعزولة شبكياً
Cryptography	PBKDF2-HMAC & RFC-7519	Native hashlib / hmac	تجزئة كلمات المرور (260 ألف دورة) وتوقيع توكنات JWT
Testing Engine	Pytest + AnyIO	>= 8.0.0	تنفيذ الاختبارات الشاملة (E2E Integration Test Suite)

2. المعمارية الهندسية ذات الطبقات الأربع (Four-Layer System Architecture)

تم بناء المنظومة على هيكلية طبقية مفصولة المسؤوليات (Separation of Concerns)، بحيث تنعزل شبكات المهاجمين المعزولين كلياً عن شبكات وتطبيقات الإنتاج:



شكل (1): المعمارية الهندسية ذات الطبقات الأربع وتدفق حركة المرور الشرعية مقابل المحتجزة

2.1 تفصيل الطبقات الهندسية الأربع

الطبقة الأولى: بوابة الخداع وفحص المرور (Layer 1)

المسؤول: علي (Ali) — المنفذ: 8000

تمثل نقطة الدخول والمصد الدفاعي الأول للمنظومة عبر `deception_gateway`. تقوم بالتحقق اللحظي من حالة الجلسات عبر استعلام Redis في أجزاء من الميلي ثانية، وتفحص الحمولات الواردة لكشف التهديدات وتمرير المرور الشرعي إلى Layer 2 مع عزل المهاجمين في Layer 3.

الطبقة الثانية: التطبيق الحقيقي ومحقن الشراك (Layer 2)

المسؤول: يوسف (App) وعمار (Injector) — المنفذ: 8001

تستضيف خدمات الويب المؤسسية الحقيقية والمصادقة المشددة `core_web_app`. ترتبط بها طبقة وسيطة `honey_token_injector` تحقن ترويسات وروابط خادعة وموقعة رقمياً بـ HMAC داخل ردود التطبيق الطبيعية لخداع المهاجمين.

الطبقة الثالثة: بيئة العزل ومحرك التحقيق الجنائي (Layer 3)

المسؤول: عمار (Ammar) — المنافذ: 8080 و 8090

تدير مجمع حاويات العزل السريعة `isolation_sandbox` المعزولة شبكياً، مع خادم خادع تفاعلي ومحاكي لينكس. يعمل بالتوازي معها محرك الأدلة الجنائية `forensic_extraction_engine` لتسجيل الأوامر وحزم PCAP وسلسلة العهدة المشفرة.

الطبقة الرابعة: التخزين وغرفة العمليات (Layer 4)

المسؤول: يوسف (Storage) وعمار (SOC) — المنفذ: 8500

توفر مستودعات التخزين غير المتزامنة في Redis و PostgreSQL لحفظ الأدلة وسجلات الحوادث. تغذي مباشرة لوحة تحكم مركز العمليات الأمنية `soc_dashboard` المزودة بالبث اللحظي بالـ WebSockets وخريطة MITRE ATT&CK وأزرار SOAR.

3. دورة حياة الطلب (Request Lifecycle Analysis)

يتبع النظام استراتيجية فرز حتمية مبنية على الحالة (State-Driven Deterministic Routing). يوضح الجدول والمخطط أدناه الفرق الجوهري بين تدفق طلب المستخدم الشرعي وطلب المهاجم:

مرحلة التدفق	مسار المستخدم الشرعي (Legitimate Path)	مسار المهاجم المحتجز (Trapped Attacker Path)
1. نقطة الدخول	يصل الطلب إلى البوابة: `GET /api/v1/records`	يصل الطلب إلى البوابة متضمناً شِركاً: `GET /api/v1/internal/vault-keys`
2. فحص Redis	تستعلم البوابة من `redis_cache_verifier`: الجلسة نظيفة (Clean).	تستعلم البوابة: إذا كانت الجلسة معزولة مسبقاً، تُحول فوراً بدون فحص جديد.
3. كشف التهديدات	يفحص `attack_detector` الترويسات والحمولة: لا توجد مؤشرات هجوم.	يكتشف `attack_detector` التوكن الخادع أو بصمة هجومية (SQLi/L4 Scan).
4. تصعيد الحالة	لا يحدث أي تصعيد؛ الـ Risk Score يبقى 0.0.	يُصعد الـ Risk Score إلى 100.0، والحالة إلى `MALICIOUS_TRAPPED`.
5. نشر التنبيه	لا يصدر أي تنبيه؛ استمرار الهدوء الأمني.	يُبث تنبيه فوري عبر Redis Streams إلى لوحة الـ SOC (وميض أحمر بالـ WebSocket).
6. توجيه الطلب	يُوجه الطلب إلى التطبيق الحقيقي `CORE_WEB_APP` (منفذ 8001).	يُوجه الطلب فوراً إلى الحاوية المعزولة `PREWARMED_SANDBOX` (منفذ 8080).
7. معالجة الرد	يُرجع التطبيق السجلات الحقيقية من قاعدة بيانات الإنتاج.	يُرجع الخادم الخادع بيانات مفاتيح مشفرة ومصطنعة (Decoy Vault Keys).
8. التعزيز بالشراك	يحقن `HoneyTokenInjector` ترويسات وروابط خادعة داخل الرد الشرعي.	لا يتم حقن شراك إضافية؛ بل يتم تثبيت المهاجم داخل بيئة العزل وتتبع أوامره.
9. التحقيق الجنائي	لا يتم التقاط أي أدلة جنائية.	يسجل محرك التحقيق الجنائي أوامر الشل، وحزم PCAP، وسلسلة العهدة المشفرة.

🔒 مبدأ العزل المطلق لشبكة الاحتجاز (Air-Gapped Sandbox Network)

الحاويات الخادعة داخل `sandbox_containment_net` مُعرفة في Docker Compose بخيار `internal: true`. هذا يعني هندسياً وقانونياً: 1) لا يمكن للمهاجم الخروج للإنترنت أو إطلاق هجمات DDoS. 2) لا يمكن للمهاجم الوصول إلى قاعدة بيانات الإنتاج أو سيرفر الـ SOC. 3) أي محاولة اتصال خارجي يتم اعتراضها داخلياً عبر محاكي C2 لجمع أدوات المهاجم دون تسريب أي بيانات.

4. الهيكلية التنظيمية للفريق وتوزيع المسؤوليات (Team Architecture)

تم تقسيم المشروع بدقة بين المهندسين الأربعة لضمان ملكية برمجية مستقلة لكل ملف، مع تكامل وظيفي محكم:

المهندس	المسمى التخصصي	المكونات والملفات المملوكة	المهام الرئيسية والتسليمات

سجاد (Sadjad)	Team & Leader System Architect	`run_all.py`
		`main.py`
		`docker-compose.yml`
		`START_PROJECT.bat`
		`tests/test_architecture_e2e.py`
		`shared_contracts/`
			▪ القيادة المعمارية وتنسيق الخوادم الخمسة.
			▪ إدارة البنية التحتية والشبكات المعزولة.
			▪ إدارة ومراجعة جودة الكود والدمج عبر Git.
			▪ الإشراف على جناح الاختبارات الشاملة.
			▪ قيادة سيناريو العرض الحي أمام اللجنة.

علي (Ali)	Backend & Deception Gateway Engineer	`deception_gateway/main.py`
		`deception_gateway/gateway_router.py`
		`deception_gateway/traffic_classifier.py`
		`deception_gateway/attack_detector.py`
		`deception_gateway/malicious_session_router.py`
		`deception_gateway/redis_cache_verifier.py`
			▪ حراسة البوابة وتصنيف حركة المرور اللحظي.
			▪ كشف هجمات SQLi والمسارات الحساسة وأدوات الفحص.
			▪ تطوير بروتوكول `TCPL4DeceptionProtocol`.
			▪ توجيه الجلسات المحتجزة والخداع المدمج البديل.

يوسف (Yousif)	Core Application & Data Engineer	`core_web_app/main.py`
		`core_web_app/legitimate_request_routes.py`
		`core_web_app/authentication/*`
		`core_web_app/database_connector/*`
		`data_storage/postgresql_store/*`
		`data_storage/redis_store/*`
			▪ بناء التطبيق الحقيقي ومسارات الأعمال المؤسسية.
			▪ تشفير PBKDF2 (260 ألف دورة) وتوليد رموز JWT.
			▪ عزل جداول الإنتاج عن جداول الأدلة الجنائية.
			▪ إدارة قواعد بيانات PostgreSQL و Redis ومحركات Fallback.

عمار (Ammar)	Honeypot, & Sandbox Forensics Engineer	`honey_token_injector/*`
		`isolation_sandbox/*`
		`forensic_extraction_engine/*`
		`soc_dashboard/*`
		`templates/index.html`
			▪ توليد وتدقيق الشراك الخداعية الموقعة بـ HMAC.
			▪ مجمع حاويات العزل السريعة بزمن استجابة صفر.
			▪ فك تعتيم الأوامر ومطابقة MITRE ATT&CK.
			▪ اقتناص PCAP ومجسات eBPF وبناء سلسلة العهدة.
			▪ تطوير لوحة مركز العمليات ومصيدة البحث في المتجر.

5. استراتيجية الاختبارات والتأهيل ليوم المناقشة النهائي

5.1 نتائج جناح الاختبارات الشاملة (E2E Verification Evidence)

تم اختبار النظام برمجياً عبر الأمر المعتمد `pytest` واجتاز بنسبة 100%:

```
============================= test session starts ============================== platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 rootdir: C:\Users\Marhaba\Downloads\fastapi-deception-architecture-main configfile: pytest.ini collected 5 items fastapi-deception-architecture-main\tests\test_architecture_e2e.py ..... [100%] ========================= 5 passed, 1 warning in 8.62s ==========================
```

5.2 سيناريو العرض الحي أمام لجنة المناقشة (Live Defense Script)

1. **التشغيل الفوري**: فتح المشروع والضغط على `START_PROJECT.bat` لعرض تشغيل الخدمات الخمس المتزامنة.
2. **استعراض الهدوء الأمني**: فتح لوحة الـ SOC على `http://127.0.0.1:8500/admin/soc` لإظهار العدادات الصفرية.
3. **استعراض الشرك الخادع**: فتح `http://127.0.0.1:8000/api/v1/records` وإظهار الترويسات التي حُقن فيها التوكن الخادع `X-Internal-Debug-Token`.
4. **شن الهجوم واستدراج المهاجم**: فتح الرابط الخادع `http://127.0.0.1:8000/api/v1/internal/vault-keys`، وإظهار وميض الإنذار الأحمر في لوحة الـ SOC عبر الـ WebSocket في نفس اللحظة!
5. **تنفيذ أمر شل في العزل**: طلب الشل `http://127.0.0.1:8000/api/v1/internal/debug-console?cmd=whoami%20%26%26%20cat%20/etc/shadow` وإظهار الرد الخادع الواقعي.
6. **الإثبات القانوني لسلسلة العهدة**: فتح مسار تدقيق العهدة وإظهار تحقق التوقيع الرياضي `overall_integrity_verified: true` واستعراض ملف الـ PCAP وملف الـ eBPF الناتجين.